

Ejercicio 1. Entorno de trabajo: control de versiones y contenedores

Parte A

- **Sistemas de control de versiones**

Los SCV son herramientas las cuales nos permiten tener un control e historial sobre los cambios que se realicen en archivos de software, lo cual facilita la colaboración y recuperación de versiones anteriores, a lo cual esto nos ayuda a que en el momento en el que estemos trabajando en dicho archivo, podamos ir probando modificaciones sin miedo a que algo salga mal y no saber cuál fue el problema y sobre todo no perder la versión anterior ante este cambio, así mismo podemos trabajar en conjunto en un equipo de trabajo y nos ayuda a que no todo se rompa o se pierda si algún integrante comete alguna error. Lo cual también cualquier cambio realizado se guardará en el historial, junto con el nombre y la fecha, así se podrá saber quien y cuando hizo dicha modificación.

- **Git y GitHub**

Al principio se podría pensar que hacen exactamente lo mismo, y casi pero no totalmente, ya que git se puede ejecutar de manera local, lo cual nos permite seguir usando sus funciones aun sin internet, lo que nos ofrece git el historial como antes mencionado, poder regresar o revisar anteriores versiones, y si estamos trabajando con un equipo de trabajo se pueden realizar pull o Push, para que todos puedan estar trabajando en un mismo proyecto de manera simultáneamente.

Pero no podremos ver lo que han trabajado o que están haciendo nuestros compañeros en tiempo real, por lo que si están trabajando en lo mismo no podrán enterarse hasta que hayan modificado el archivo, lo cual podría verse que hay algunos pequeños inconvenientes al momento de trabajar con un equipo en git, pero si uno quiere trabajar de manera individual dichos inconvenientes no se presentan.

Con GitHub, estos problemas desaparecen ya que GitHub facilita la manera en la que se colabora, ya que aquí se pueden observar en lo que nuestros compañeros están trabajando en tiempo real, también nos incluye funciones de organización y gestión de proyectos, por lo que podremos asignar tareas a una persona o a varias personas en conjunto, aparte cualquier repositorio que hagamos estará publico lo que esto hará que cualquier persona que quiera interactuar o ayudar con dicho código, podrá

hacerlo con un pull para que el autor de dicho repositorio poder observar que cambios hizo y si así lo desea permitir los cambios.

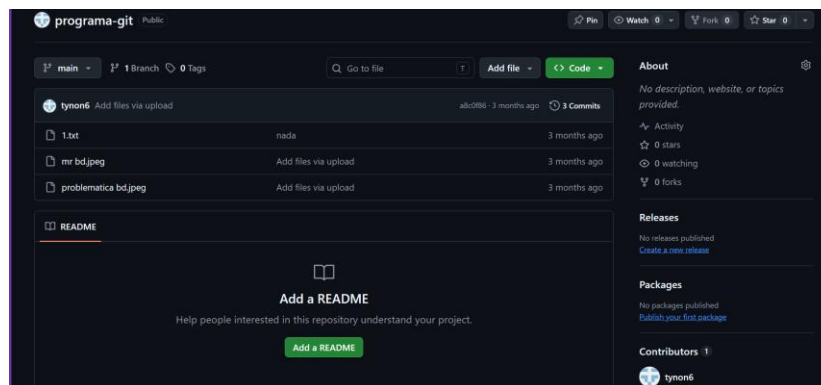
En resumidas cuentas, Git es un software de control de versiones local, lo cual para proyectos individuales esta perfecto.

GitHub, es una plataforma web que tiene características de git, pero hacia un enfoque más colaborativo.

- **Repositorio, Commit, Branch, merge, conflicto de fusión, Pull request, Archivo .gitignore y archivo README.**

Repositorio:

Este es el elemento más básico de GitHub y permite a los desarrolladores poder gestionar y colaborar con proyectos. Es un lugar donde puede guardar todos los archivos, códigos e historial de revisiones de cada archivo que se haya modificado, así mismo, cada repositorio puede tener varios colaboradores y ser públicos o privados.



Commit:

Un commit es como si fuese un punto de partida, o también se le conoce como “Instantánea” lo cual hace que guarde el estado actual de tu archivo, donde podrás regresar más adelante si lo crees necesario, puedes ver las diferentes versiones de tu archivo y también debe de contener un mensaje descriptivo donde expliques los cambios que hayas hecho. Al hacer un commit también lo que hace es crear un Hash que es un identificador único.

```
Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   file.txt

Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git (master)
$ git commit -m "first commit"
[master (root-commit) 5a457ea] first commit
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 file.txt

Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git (master)
$ git status
```

Branch:

Una Branch son espacios de trabajo aislados lo que quiere decir esto es que tienes una copia de tu código principal o el que deseas trabajar, y puedas probar algunas cosas que desees implementar o cambiar, y poder ver el funcionamiento o comportamiento que estos tengan, lo que significa que si algo sale mal tu código principal no saldrá afectado, esto es de gran ayuda para poder experimentar diferentes cosas sin el temor de afectarlo, y una vez que estes seguro que todo va a funcionar, puedas fusionarlo, para que todos tus cambios ahora estén en el código principal.

```
tuts@fossilinux:~/Desktop/FossilLinux-Web$ git branch FossilLinux-Branch
tuts@fossilinux:~/Desktop/FossilLinux-Web$ git branch
  FossilLinux-Branch
* master
* stage
tuts@fossilinux:~/Desktop/FossilLinux-Web$
```

Merge:

Existen 3 tipos de merge.

-Merge Commit:

Lo que hace esto es que crea un nuevo commit que une la rama base con la rama que deseas fusionar. Este commit tiene dos padres, representando el punto de unión de ambas ramas.

Las ventajas de este commit es que básicamente agrega a tu historial todo el historial de la rama secundaria, lo que permite seguir rastreando los cambios realizados, pero lo malo también es que tendrás muchos historiales lo que podría complicar un poco la lectura del proyecto, pero eso no significa que sea malo este merge, es de mucha ayuda cuando hay equipos de trabajo grandes.

-squash merge:

Cuando se realiza un squash and merge, GitHub toma todos los commits de la rama de desarrollo o feature branch y los aplasta en un solo commit que se añade a la rama principal.

-Rebase and merge:

En una solicitud de incorporación de cambios, todas las confirmaciones de la rama de tema (o rama de encabezado) se agregan a la rama base por separado sin una confirmación de combinación.

De este modo, el comportamiento de fusionar mediante cambio de base y combinar es similar a una combinación de avance rápido, ya que mantiene un historial de proyectos lineal. Sin embargo, el rebase lo logra al rescribir el historial de confirmaciones en la rama base con confirmaciones nuevas.

Conflicto de Fusión:

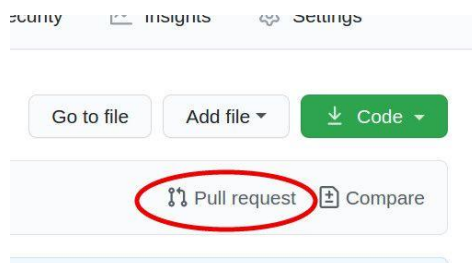
Por lo general, Git puede resolver las diferencias entre las ramas y fusionarlas automáticamente. Normalmente, los cambios se encuentran en líneas diferentes o en archivos diferentes, por lo que Git puede combinarlos sin ayuda. A veces, los cambios que entran en conflicto necesitan tu ayuda. Los conflictos de combinación a menudo se producen cuando las personas realizan cambios diferentes en la misma línea del mismo archivo, o cuando una persona edita un archivo y otra persona elimina el mismo archivo.

```
CONFLICT (content): Merge conflict in resumé.txt Automatic merge failed; fix conflicts and then commit the result.
```

Pull request:

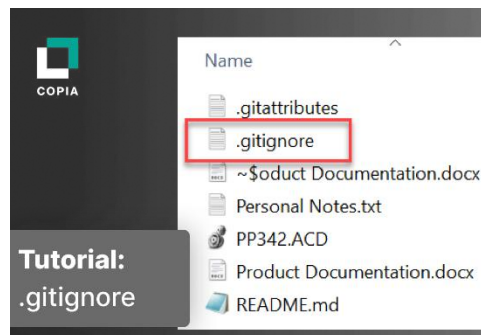
Una pull request propone fusionar cambios de código de una rama en otra. Como función colaborativa, las pull requests te ofrecen un espacio para comentar y revisar el trabajo antes de que pase a formar parte de un proyecto.

Las pull requests convierten una serie de cambios en el código en una conversación. En lugar de fusionar el trabajo directamente, lo propones para que los colaboradores puedan opinar. Esto le ayuda a usted y a su equipo a mantener código seguro y de alta calidad.



Archivo .gitignore:

Un archivo .gitignore se coloca generalmente en el directorio raíz de un proyecto y contiene patrones que le dicen a Git qué archivos o directorios no deben incluirse en el historial de cambios, su función es mantener el repositorio limpio y eficiente, asegurando que solo se rastreen los archivos esenciales del proyecto.



Archivo README:

Un archivo README sirve para comunicar información importante sobre tu proyecto. También es un documento escrito en Markdown, un lenguaje de marcado que permite dar formato al texto de manera sencilla y visualmente atractiva.

Su función principal es informar a los visitantes del repositorio sobre el proyecto, incluyendo qué hace, por qué es útil y cómo empezar a usarlo.

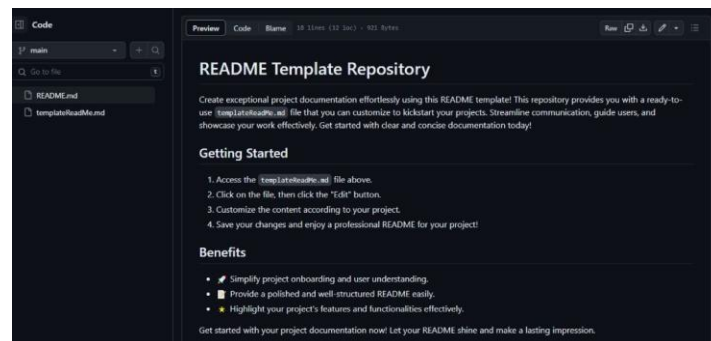


Imagen:

Es una plantilla de solo lectura que incluye el código de la aplicación, librerías, dependencias, herramientas y configuraciones necesarias para su ejecución de manera aislada, también funciona como un punto de partida para crear contenedores, que son instancias en ejecución de esa imagen.

Contenedor:

Un contenedor Docker encapsula el código de la aplicación, su tiempo de ejecución, bibliotecas, herramientas del sistema y configuraciones necesarias para que la aplicación funcione correctamente, sin depender del entorno subyacente del sistema operativo.

Cada contenedor es una instancia de una imagen Docker, lo que permite ejecutar múltiples contenedores a partir de la misma imagen, cada uno aislado y con su propio espacio de ejecución.

Volumen:

Es una carpeta o directorio creado en el host, pero administrado exclusivamente por Docker. Su ciclo de vida es independiente del contenedor, lo que garantiza que los datos no se pierdan al borrar o recrear contenedores.

Puerto Publicado:

Es un puerto específico de la máquina anfitriona (host) que se conecta a un puerto interno del contenedor, permitiendo que el tráfico externo pueda atravesar el aislamiento de Docker.

Entorno Virtual de Python:

Un entorno virtual es un directorio aislado que contiene su propio conjunto independiente de librerías y dependencias, junto con una copia o enlaces simbólicos a un intérprete de Python específico. Su propósito principal es permitir el desarrollo de diferentes proyectos sin que las dependencias de uno interfieran con las de otro o con la instalación global del sistema.

¿Por qué un entorno virtual no modifica la versión del intérprete?

Al crear un entorno virtual (por ejemplo, utilizando el módulo nativo venv), este se genera tomando como **base exacta** el binario del intérprete con el que se ejecutó el comando de creación. No descarga ni instala un motor de Python diferente, así mismo el objetivo del entorno virtual es gestionar las dependencias de terceros (como librerías instaladas vía pip), no alterar las características del lenguaje ni actualizar el núcleo de Python en tu sistema operativo.

¿Por qué conviene fijar la versión de la imagen, python:3.12-slim, en lugar de emplear python:latest.?

Conviene fijar una versión específica como python:3.12-slim en lugar de usar latest principalmente por motivos de **reproducibilidad y estabilidad** ya que a etiqueta latest se actualiza automáticamente cada vez que se libera una nueva versión o parche de Python. Si compilas tu imagen hoy y la vuelves a compilar en el futuro, podrías obtener un entorno diferente sin darte cuenta, lo que puede romper tu aplicación de manera imprevisible, **y control de versiones** porque al especificar 3.12, te aseguras de que el comportamiento del intérprete, las bibliotecas base y la compatibilidad de tu código se mantengan exactamente iguales en los entornos de desarrollo, pruebas y producción.

PARTE B

[tynon6/Practica-1](#)

Link del repositorio, las imágenes sobre este punto estarán en el repositorio.

PARTE C

Explicación de lo que hacen las líneas del **Dockerfile**.

- **ARG PYTHON_VERSION=3.12:** Define una variable para parametrizar y seleccionar la versión de Python al momento de construir la imagen (por defecto usa la 3.12).
- **FROM python:\${PYTHON_VERSION}-slim:** Selecciona la imagen base oficial de Python en su versión ligera (*slim*), reduciendo el tamaño y optimizando recursos.
- **WORKDIR /app:** Establece /app como el directorio de trabajo predeterminado dentro del contenedor.
- **RUN python -m venv /opt/venv:** Crea un entorno virtual aislado ubicado en /opt/venv, separado de la aplicación tal como lo solicita la nota de diseño.
- **ENV PATH="/opt/venv/bin:\$PATH":** Modifica la variable de entorno PATH para asegurar que se utilicen los binarios del entorno virtual creado.
- **RUN pip install --no-cache-dir --upgrade pip \ && pip install --no-cache-dir -r requirements.txt:** Actualiza el gestor de paquetes pip evitando almacenar caché para mantener la imagen limpia.
- **CMD ["Python", "src/app.py"]:** Define el comando por defecto que se ejecutará al iniciar el contenedor.

Explicación de lo que hacen las líneas del **compose.yml**

- **services:** : Bloque donde se declaran los contenedores o servicios independientes que gestionará Docker Compose.
- **py311, py312, py313:** Identificadores únicos para cada servicio correspondientes a sus respectivas versiones de Python.
- **build:, context: . :** Indica que la construcción de la imagen se realizará utilizando el contexto de la carpeta actual.
- **args: PYTHON_VERSION: "3.1X":** Envía el argumento de la versión específica (3.11, 3.12 o 3.13) hacia el Dockerfile.
- **volumes: - ../app:** Monta el directorio del proyecto del host dentro del contenedor en la ruta /app, permitiendo la persistencia y sincronización de archivos.
- **command: python --version:** Ejecuta la instrucción para verificar y reportar la versión del intérprete al correr el servicio.

